a class—for example the methods that you want to group together may be utility methods that don't manage their own state.

An initial approach may be to put all these things into a file and simply load that file into any program that needs it. This is the way the C language works. However, this approach has a potential problem—name collisions. To give an admittedly contrived example, say you write a set of the trigonometry functions, sin, cos, and so on. You stuff them all into a file, trig.rb, for future generations to enjoy. Meanwhile, another developer is working on a role-playing game where characters might choose to be good or evil, and codes a set of her own useful routines, including be_good and sin, and sticks them into a file called moral.rb. Now you want to add some physics calculations to this game, and you therefore, you need to load both trig.rb and moral.rb into your program. But both define a method called sin. Bad news.

The answer is the module mechanism. Modules define a *namespace*, a sandbox in which your methods and constants can play without having to worry about being stepped on by other methods and constants. The trig functions can go into one module:

tut_modules/trig.rb

```
module Trig
  PI = 3.141592654
  def self.sin(x)
    # ..
  end

  def self.cos(x)
    # ..
  end
end
```

and the good and bad "moral" methods can go into another:

tut_modules/moral.rb

```
module Moral
  VERY_BAD = 0
  BAD = 1
  def self.sin(badness)
    # ...
  end
end
```

Module names are just like class names, both are global constants with an initial uppercase letter. Their method definitions look similar, too: module methods are defined just like class methods, using the def self.method_name syntax.

If a third program wants to use these modules, it can simply load the two files (using the Ruby require or require_relative statement). To reference the name sin unambiguously, our code can then qualify the name using the name of the module containing the implementation we want:

tut_modules/pin_head.rb

```
require_relative "trig"
require_relative "moral"
y = Trig.sin(Trig::PI / 4)
wrongdoing = Moral.sin(Moral::VERY_BAD)
```